Báo cáo tuần 1
Xv6 và luồng I/O virtio

Nguyễn Văn Hoàng • MSSV 20233413
Đề tài 26128 - Asynchronous virtio-blk and I/O schedulers

Kỳ công việc được kiểm chứng: **23/10/2026** (UTC+7). Báo cáo ghi lại ba nhiệm vụ cá nhân: clone và chạy xv6; phân tích virtio_disk_rw, vòng đời descriptor và interrupt; vẽ sequence submit/complete bất đồng bộ. Đây là phụ lục kỹ thuật cá nhân phục vụ báo cáo nhóm và demo.

Kết quả đã làm

Nhiệm vụ	Kết quả và giới hạn
Clone và chạy xv6	Repo đã clone và chuyển vào 2026.1/HDH. Rebuild, boot, shell và kiểm tra file trên CPUS=1/4 thành công.
Phân tích driver và interrupt	Đã đối chiếu dòng code, hai wait channel, rollback khi thiếu descriptor, lost-wakeup và đường panic. Chưa có trace/injection cho các nhánh lỗi.
Sequence async	Có hai sơ đồ stock/async đã đồng bộ bố cục draw.io người dùng chỉnh sửa, kèm Mermaid và SVG. Async là đề xuất, chưa có thay đổi driver hoặc syscall.

Môi trường và phiên bản

Thông tin	Giá trị thực tế
Hệ điều hành	Ubuntu 24.04.3 LTS x86_64; Linux 7.2.6
RISC-V toolchain	riscv64-unknown-elf-gcc 13.2.0; GNU ld 2.42
QEMU và công cụ	QEMU 8.2.2; GNU Make 4.3; Python 3.12.3
Project commit	a0cfa2e942fd7cf27040ec404b002e5a25c691d9
Xv6 pin	f780d7850765d74cbdcd1fff4b822d8829f7c9bd

Trạng thái Git: chưa tạo commit, chưa push. Kernel, user, mkfs, Makefile và mk không đổi. Pin giảng viên và review của Hiếu còn chờ xác nhận; không coi đây là nghiệm thu toàn bộ tuần 1 của nhóm.

1 Chạy lại và đối chiếu bằng chứng

```
cd /home/ubuntu/2026.1/HDH
cd Asynchronous-virtio-blk-and-IO-Schedulers
git rev-parse HEAD
cat .xv6-commit
bash scripts/check-env.sh
make test-fast
python3 scripts/check-week01-hoang.py
make qemu CPUS=1
```

Trong shell xv6, dùng **echo HOANG_DEMO > hdhdemo**, **cat hdhdemo**, **sync** và **ls**. Thoát QEMU bằng Ctrl-A rồi X; mở lại cùng fs.img và đọc file để demo dữ liệu sau reboot. Không chạy make clean hoặc test fresh_fs giữa hai lần boot cần giữ file.

Kết quả thực tế ngày 23/10

Kiểm tra	CPUS	Kết quả
Rebuild kernel/kernel + fs.img	-	PASS, exit 0
write-read-cpus1	1	PASS, 1.460s
reboot-read-cpus1	1	PASS, 1.646s
write-read-cpus4	4	PASS, 1.458s
reboot-read-cpus4	4	PASS, 1.647s
boot-smoke	1	PASS, 0.85s
shell-fs-cpus3	3	PASS, 1.26s

Tổng theo phạm vi: build thành công; helper có **4/4 smoke PASS** trên CPUS=1/4; stock-fast có **2/2 case PASS** trên CPUS=1/3. env-check cũng boot PASS nhưng không cộng lượt boot lặp vào số case mới. Thời gian trên là thời gian harness, không phải latency I/O hay benchmark.

```
xv6 kernel is booting
init: starting sh
$ cat hcheck
HOANG_W01_CPUS4_20261023
$ sync
```

Trích serial thật từ reboot-read-cpus4.log. Helper gọi sync trước kết thúc QEMU rồi boot lại cùng image: kiểm tra lưu dữ liệu qua reboot của guest, không chứng minh an toàn khi mất điện host. Helper tạo mới fs.img cho mỗi cấu hình CPU và sẽ thay dữ liệu demo trong image.

NOT RUN: full usertests/grind, stress 30 phút, max_inflight, descriptor exhaustion, error injection và GDB trace. Cảnh báo môi trường: thiếu gdb-multiarch và clang-format; không ngăn build/boot. Log baseline máy dev đã có trong repo không được nhận là kết quả mới trên máy này.

2 Phân tích virtio_disk_rw

Contract hiện tại: caller truyền buffer có lifetime ổn định và đang giữ buf sleeplock. virtio_disk_rw(b, write) thực hiện một block I/O rồi chờ hoàn tất; đây là API đồng bộ với caller. bread chỉ gọi khi cache miss, còn bwrite hiện do log gọi. [C1, C2]

Bước	Code và ý nghĩa
1 • Địa chỉ	virtio_disk.c:218 tính sector = blockno × (BSIZE/512). BSIZE=1024, nên block 17 bắt đầu tại sector 34, độ dài truyền 1024 byte.
2 • Tài nguyên	Dòng 220-236 giữ vdisk_lock và cấp ba descriptor. Thiếu chỗ thì alloc3 rollback phần dở, prepare trên &disk.free[0], nhả khóa, ngủ rồi retry.
3 • Chain	Dòng 241-268 tạo header tại ops[head], data tại b.data và status tại info[head].status. Status ban đầu 0xff. Ba descriptor không cần liền nhau.
4 • Publication	Dòng 270-284 đặt b.disk=1 và info[head].b, ghi avail slot, fence, tăng avail.idx, fence, notify queue 0. Map phải sẵn trước publication.
5 • Wait	Dòng 287-292: while b.disk==1, sleep_prepare(b), release, sleep, acquire. Caller không giữ vdisk spinlock khi ngủ.
6 • Reclaim	Dòng 294-297: caller xóa info[head], free_chain và nhả khóa. bread sau đó mới set valid=1; tầng dùng buffer tự brelse.

Ba descriptor của một request stock

Vùng	Địa chỉ và độ dài	READ flags	WRITE flags
Header	&ops[head], 16 byte	NEXT = 1	NEXT = 1
Data	b.data, 1024 byte	NEXT | WRITE = 3	NEXT = 1
Status	&info[head].status, 1 byte	WRITE = 2	WRITE = 2

Bit WRITE theo góc nhìn thiết bị: device được phép ghi RAM, không phải user yêu cầu ghi đĩa. READ cần device ghi vào data; WRITE để device đọc data. Header chứa type IN=0/OUT=1, reserved=0, sector. NEXT nối chain, status là phần tử cuối. [C1, C3]

io_fence trong riscv.h:370-375 là fence iorw, iorw kèm compiler memory clobber. Hai fence bảo đảm thứ tự descriptor/avail trước idx và notification; spinlock giữa CPU không thay cho quy tắc publication với thiết bị. [C4]

3 Descriptor lifecycle và điều kiện chờ

```
FREE -> RESERVED -> CHAIN READY -> PUBLISHED
     -> DEVICE COMPLETED -> REAPED BY IRQ
     -> FREED BY CALLER -> FREE
```

Tên trạng thái trên phục vụ phân tích, không phải enum request đã có trong stock. free_desc xóa addr/len/flags/next rồi đánh dấu free; free_chain phải đọc flags/next trước khi xóa phần tử hiện tại. alloc3_desc trả lại phần đã cấp dở nếu không đủ ba descriptor. [C1:155-213]

Hai chain là giới hạn tài nguyên

NUM=8 và chi phí 3 descriptor/request cho giới hạn **floor(8/3)=2 chain**. Khi hai chain giữ 6 descriptor, caller tiếp theo chỉ còn 2; alloc3 rollback rồi chờ. Chain đã complete nhưng caller chưa chạy lại vẫn chiếm descriptor. Đây là suy luận tĩnh, chưa đo max_inflight; caller đồng bộ không đồng nghĩa toàn driver chỉ có một outstanding.

Channel	Chờ điều gì	Ai wakeup
&disk.free[0]	Không cấp đủ 3 descriptor	free_desc trong caller reclaim hoặc rollback
b	b.disk còn 1	IRQ handler sau used entry thành công

Khóa và lost wakeup

bcache.lock bảo vệ lookup/refcnt/LRU; buf.lock là sleeplock; vdisk_lock bảo vệ descriptor, ring bookkeeping, info và b.disk. **bget nhả bcache.lock trước acquiresleep(buf.lock)**, không có ba khóa lồng nhau liên tục. Caller giữ buf sleeplock khi vào driver và chỉ nhả vdisk_lock để ngủ. IRQ không nhả sleeplock của caller. [C2, C5]

```
Caller: lock -> check b.disk -> sleep_prepare(b)
Caller: release(vdisk_lock)
IRQ:    b.disk=0; wakeup(b) -> p.chan=0
Caller: sleep() sees chan==0 -> returns immediately
Caller: acquire(vdisk_lock) -> recheck while
```

Nếu process đã ngủ, wakeup đặt SLEEPING thành RUNNABLE. Nếu wakeup tới trước sleep, p.chan đã bằng 0 nên sleep không đưa process vào ngủ. Phải prepare khi còn giữ khóa bảo vệ điều kiện, rồi mới release; luôn recheck bằng while. [C5:547-595]

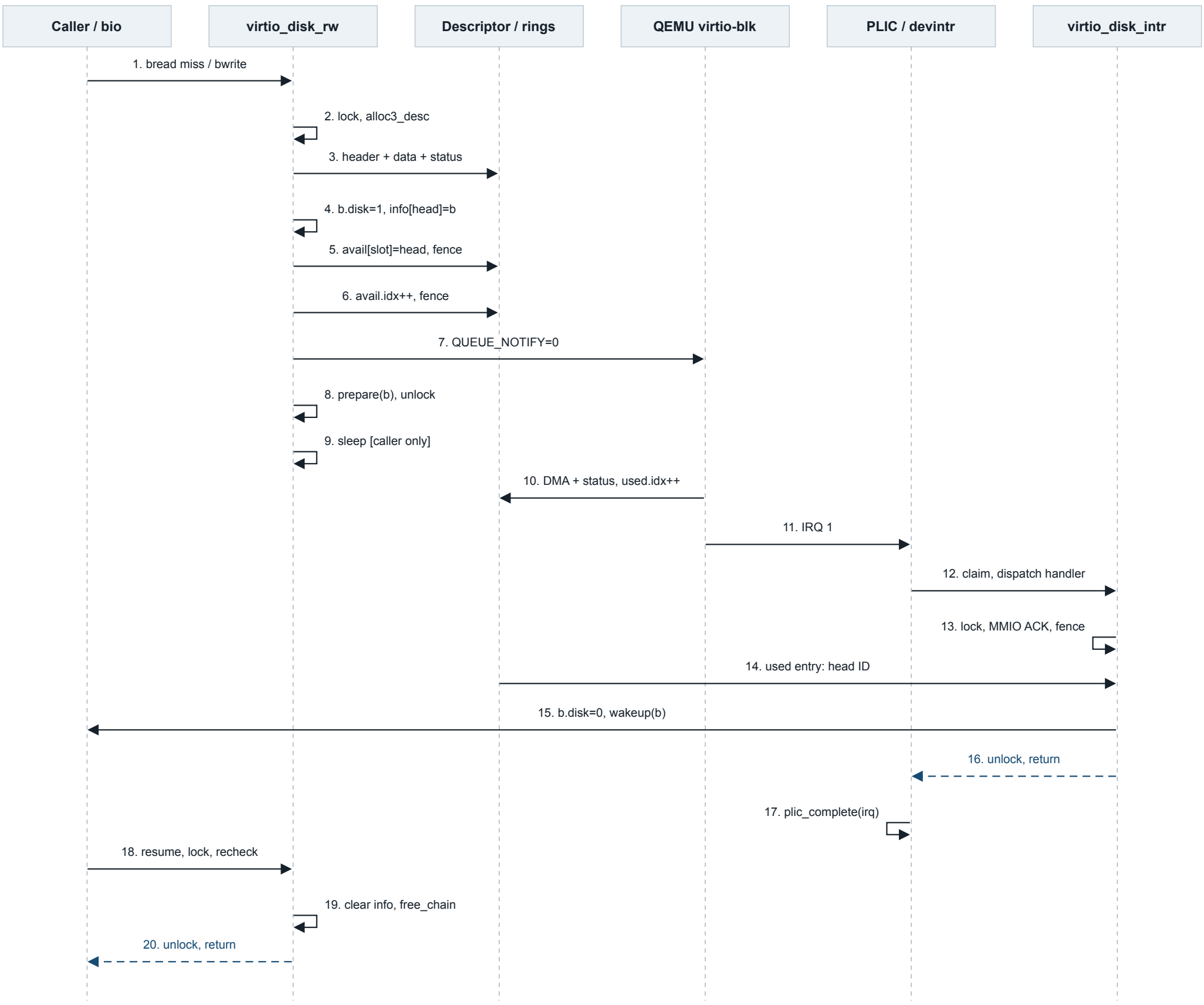
Nhánh chưa thử động	Hành vi stock và điểm cần làm sau
status != 0	panic ở IRQ; async cần ERROR, trả lỗi và cleanup một lần.
Device không complete	Không có timeout; không được free DMA chỉ vì caller timeout.
Free sai / used id hỏng	free_desc có panic; used ID vẫn được tin, cần validation trong thiết kế mới.

4 Sequence của luồng stock đã đối chiếu code

Caller chờ completion; IRQ chỉ báo hoàn tất; caller reclaim descriptor. Hình là một lịch thực thi hợp lệ, không phải trace GDB hay số đo overlap.

Bố cục xuất trực tiếp từ bản draw.io người dùng chỉnh sửa; trang sơ đồ dùng khổ A3 để giữ nhãn dễ đọc.

Luồng submit và complete trong xv6 stock

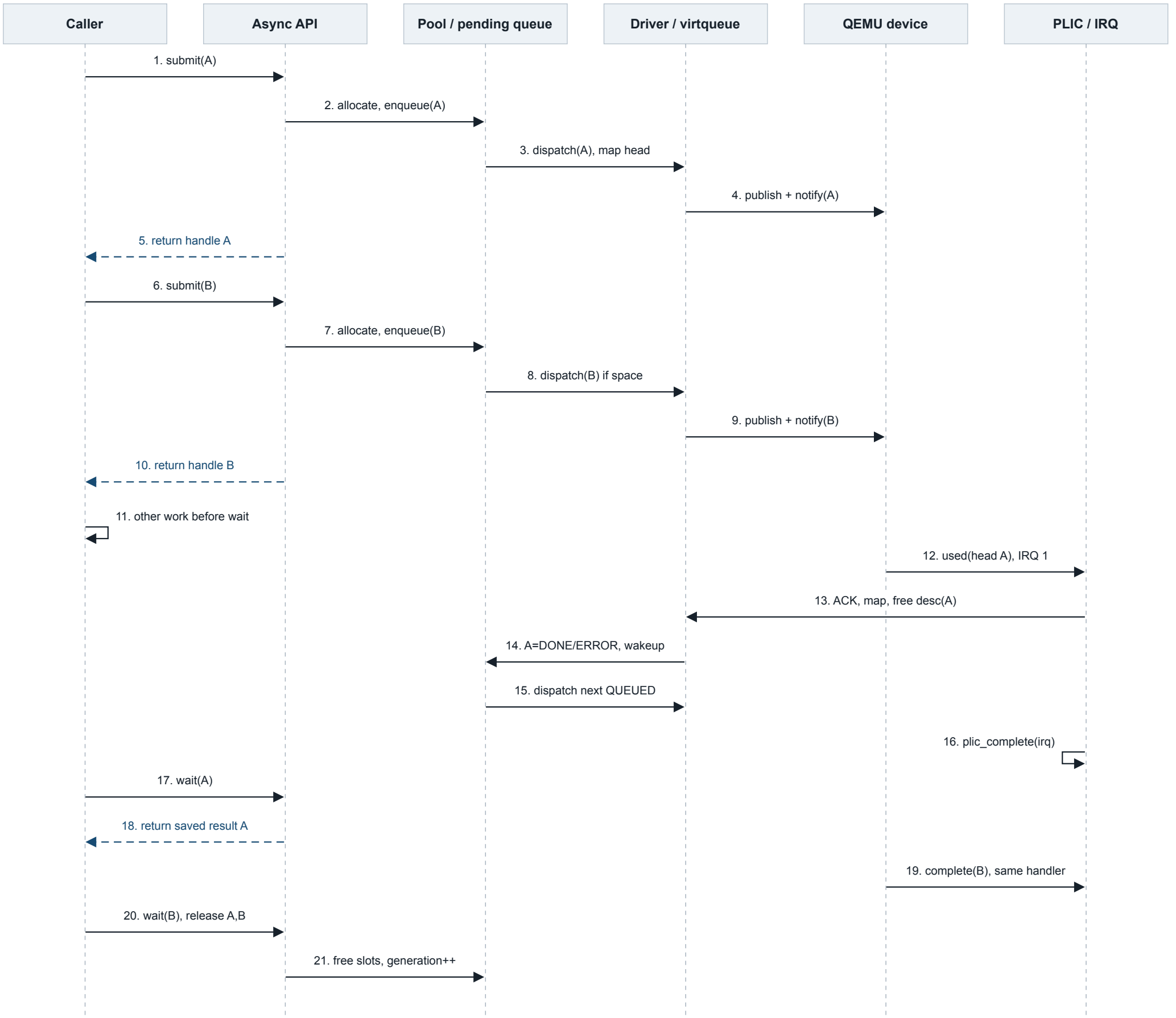


Nguồn: C1 virtio_disk.c:216-333; C5 proc.c:547-595; C6 trap.c:188-220. MMIO ACK nằm trong IRQ handler; PLIC complete nằm ở devintr sau khi handler trả. Dòng prepare phải trước unlock; nếu completion tới trước sleep thì sleep trả ngay.

5 Sequence submit và complete bất đồng bộ đề xuất

DESIGN ONLY • Submit trả handle trước wait. Request/result/buffer còn sống đến release; descriptor được reclaim trong completion đề xuất. Đây chưa phải hành vi kernel đã triển khai.

Bố cục giữ nguyên bản web; phần chú thích thiết kế được đặt ngoài hình để không sửa lại sơ đồ của người dùng.



Pool, queue, state và head map dùng khóa chung ở bản thiết kế đầu. Nếu chưa đủ descriptor, request giữ QUEUED; completion gọi dispatch tiếp. Sơ đồ minh họa hai submit trước wait; chỉ trace/counter mới chứng minh max_inflight >= 2. “Same handler” ở completion B lặp lại ACK, map, reclaim, wakeup và PLIC complete.

6 Interrupt path và contract async

Interrupt path stock: device cập nhật data/status/used rồi phát IRQ 1. CPU đi qua uservec/usertrap hoặc kernelvec/kerneltrap, cùng vào devintr. scause=0x8000000000000009 chỉ supervisor external interrupt; devintr claim PLIC, IRQ 1 gọi virtio_disk_intr. [C6, C7]

Handler lấy vdisk_lock, ACK INTERRUPT_STATUS & 0x3, fence rồi drain mọi used entry chưa xử lý. Entry id là head descriptor: tìm info[id], kiểm tra status, đặt b.disk=0, wakeup, tăng used_idx. Sau khi handler trả, devintr mới plic_complete. Một IRQ có thể xử lý nhiều completion; wakeup không có nghĩa caller chạy ngay.

Phân tách hai vòng đời trong đề xuất

Tài nguyên	Đề xuất ai quản lý	Khi được tái sử dụng
Descriptor	Dispatcher cấp; IRQ reclaim sau complete thật	Sau khi device đã hoàn tất và clear head map
Request / handle	Bounded pool với id + generation	Sau DONE/ERROR, caller release và hết reference
DMA buffer / header / status	Kernel memory ổn định, chuẩn bị trước	Không còn DMA và consumer đã dùng xong kết quả

aio_read_submit validate, try-allocate, enqueue và dispatch nếu có chỗ, rồi trả handle. Pool đầy có thể trả EAGAIN; request chưa có descriptor ở QUEUED. aio_poll đọc state; aio_wait chờ trong process context; aio_release thu hồi slot cuối cùng. Tên API/errno còn chờ review, không phải syscall đã tồn tại.

```
aio_wait(handle):
  acquire(io_lock)
  req = validate_handle(handle)
  if invalid: release(io_lock); return error
  while req.state in {QUEUED, INFLIGHT}:
    sleep_prepare(req)
    release(io_lock); sleep(); acquire(io_lock)
  result = req.status
  release(io_lock)
  return result   // request release is separate
```

Trong completion đề xuất: ACK/fence; lấy head; validate map; đọc status; clear map; free_chain; tăng used_idx; ghi DONE/ERROR; wakeup; dispatch tiếp khi có chỗ. IRQ không malloc/block, copyout hoặc nhả sleeplock caller. Wrapper đồng bộ submit -> wait -> release phải giữ ordering của log; chưa chốt write fence thì chưa thể khẳng định crash consistency.

7 Hướng dẫn demo và trả lời câu hỏi

Thời gian	Nội dung trình bày và bằng chứng
0-1 phút	Nêu ba nhiệm vụ, commit project, xv6 pin và phạm vi tuần này; phân biệt kết quả chạy thật với thiết kế.
1-3 phút	make qemu CPUS=1; echo/cat/sync/ls. Mở checks.json và serial log reboot; giải thích 4/4 smoke và 2/2 stock-fast.
3-5 phút	Mở virtio_disk.c:216 và virtio.h:53. Giải thích header/data/status, bit WRITE và hai wait channel.
5-7 phút	Mở trap.c:188, virtio_disk.c:301 và proc.c:547. Dùng hình stock nói ACK, PLIC, wakeup, caller reclaim và lost-wakeup.
7-9 phút	Dùng hình async: submit trả handle, giữ buffer lifetime, IRQ reclaim descriptor, DONE/ERROR và dispatch request đang queued.
9-10 phút	Nêu giới hạn kiểm thử, review/pin GV còn pending; trình bày K02 tuần sau với DoD checksum đúng và không rò descriptor.

Các câu cần tự giải thích được

Câu hỏi	Trả lời ngắn
READ sao có WRITE flag?	WRITE cho device ghi RAM; khác OUT ghi đĩa.
Stock chỉ một outstanding?	Không suy vậy: caller nhả spinlock để ngủ, process khác có thể submit; giới hạn 2 chain từ code, chưa đo mức thực tế.
Ai free descriptor?	Stock là caller sau wakeup; IRQ reclaim là thay đổi được đề xuất cho async.
Complete trước wait có treo?	Prepare dưới khóa đăng ký channel, wakeup xóa channel; sleep thấy đã wake thì trả ngay, vẫn recheck while.
Async đã chạy chưa?	Chưa. Có sequence, contract và pseudocode; chưa có kernel code/API mới.
Đọc lại file chứng minh gì?	Cùng phiên có thể hit cache; reboot kiểm tra dữ liệu image. Không suy thành crash test hay benchmark.

Trước buổi báo cáo, bạn nên tự chạy demo một lượt và chỉ vào đúng dòng code khi giải thích. Không đọc nguyên báo cáo PDF; dùng sơ đồ để kể vòng đời và mở log để chứng minh kết quả.

8 Nội dung gợi ý để điền báo cáo tiến độ

Spec yêu cầu báo cáo tuần **một trang**, summary **200-300 từ** và theo các mục của Progress Report template.docx. PDF này là phụ lục cá nhân, chưa phải báo cáo nhóm hoàn chỉnh. Đoạn dưới chỉ mô tả phần việc Hoàng, không tự xác nhận kết quả Hiếu/Lợi hoặc review của nhóm. [S1, S2]

Trong ngày 23/10/2026, phần việc cá nhân tập trung vào môi trường xv6, khảo sát virtio và thiết kế submit/complete bất đồng bộ cho đề tài 26128. Repository được clone và đặt trong thư mục làm việc 2026.1/HDH. Mã được kiểm tra tại project commit a0cfa2e và xv6 pin f780d785, chưa thay đổi kernel.

Trên Ubuntu 24.04.3, toolchain RISC-V GCC 13.2.0 và QEMU 8.2.2 đã build, boot và chạy shell thành công. Bốn kiểm tra đọc/ghi và đọc lại sau reboot trên CPUS=1/4 đều PASS; hai case stock-fast cũng PASS. Mỗi lần chạy có metadata và serial log. Đây là smoke test, chưa thay thế usertests/grind hoặc stress 30 phút.

Phân tích code xác định mỗi request dùng ba descriptor, hai kênh chờ riêng cho tài nguyên và completion, cùng trình tự sleep_prepare để chống mất wakeup. Interrupt xử lý used ring, báo hoàn tất và đánh thức caller; trong stock, caller mới giải phóng descriptor. Hai sequence diagram mô tả luồng hiện tại và phương án async dùng request pool, handle cùng buffer có lifetime ổn định.

Đầu ra cá nhân đã có bản đồ code, sơ đồ và hướng dẫn tái lập; chưa có triển khai async hoặc commit mới. Pin chính thức và review còn pending. Bước tiếp theo là thống nhất interface với nhóm, triển khai K02 và kiểm thử checksum, completion-before-wait, cleanup descriptor; kết quả hai thành viên còn lại cần được bổ sung vào báo cáo nhóm.

Đoạn gợi ý có 243 từ khi đếm theo khoảng trắng. Nếu kỳ báo cáo bao phủ cả tuần, nhóm phải bổ sung kết quả thực tế các ngày khác; không đổi ngày cho đủ tuần nếu không có bằng chứng.

Các mục còn lại trong mẫu

Mục	Cách điền trung thực
Personal contribution	Hoàng: environment, io-path, sequence, helper và log. Ghi “chưa commit” ở cột commit/PR.
Developed code review	Kernel/user không sửa; helper kiểm chứng mới; driver async và scheduler chưa triển khai.
TODO và milestone	K02 đề xuất 8 giờ, phụ thuộc interface K01/K03; DoD một request checksum đúng, head mapping và reclaim không rò. Không coi 8 giờ là giờ công đã làm.

9 Nguồn đối chiếu và bàn giao

Nguồn yêu cầu và mã nguồn

Mã	Nguồn và phần đã dùng
S1	Doc/Spec/Xv6-riscv Term Projects.docx: báo cáo tuần một trang, đóng góp cá nhân, testing standards và nguyên tắc AI theo chính sách học phần.
S2	Doc/Spec/Progress Report template.docx: summary 200-300 từ, technical activities, personal contribution, developed code review và TODO.
S3	Doc/Spec/Project catalogue.docx, mục 26128; Doc/T1/02_Checklist_tuan_1.md, 04_Backlog_va_phan_cong.md, 03_De_xuat_thiet_ke_26128.md, 06_Huong_dan_bao_cao_tuan_1.md.
C1-C3	C1 kernel/virtio_disk.c:155-333; C2 kernel/bio.c:58-137; C3 kernel/virtio.h:53-99 và fs.h:5, ở project commit a0cfa2e.
C4-C7	C4 kernel/riscv.h:370-375; C5 proc.c:547-595; C6 trap.c:188-220; C7 plic.c:12-47 và memlayout.h:25-26.
V1	OASIS VIRTIO 1.2 mục 2.7 (split virtqueue), 4.2 (MMIO), 5.2.6 (block operation), đối chiếu 23/10/2026. Code local là căn cứ cho hành vi của pin này.

Mở đặc tả VIRTIO 1.2 chính thức. Nguồn ngoài được dùng để kiểm tra thuật ngữ flags/sector/status, không lấy giải pháp project từ repo khác.

Nguồn sơ đồ ProjectTerm - ET4290. Chỉ lấy hai trang sequence; 20 bước stock và 21 bước async không đổi nội dung. drawio-source.json ghi page ID và checksum.

File đã thêm hoặc cập nhật

```
docs/week01/io-path.md          # phan tich da doi chieu
docs/week01/async-design.md     # contract va pseudocode
docs/week01/hoang.md            # chay lai, demo, Q&A
docs/week01/environment.md      # dong may Hoang + log
docs/week01/README.md           # lien ket dau ra Hoang
docs/week01/diagrams/*.drawio   # ban chinh dong bo tu web
docs/week01/diagrams/*.mmd      # thu tu thong diep tuong duong
docs/week01/diagrams/*.svg      # anh vector
docs/week01/reports/*.pdf       # bao cao trong repo
docs/week01/git-workflow-hoang.md # commit va push
scripts/check-week01-hoang.py   # smoke CPUS=1/4 + reboot
```

Bằng chứng có thể commit nằm trong **results/week01/hoang-2026-10-23/**: checks.json, session.json, env-check.log, build.log, bốn serial log và stock-fast/{meta.json, summary.md, logs/}. Đây là bản sao log cũ, không phải lượt chạy mới. results/local vẫn bị gitignore.

Bản PDF và gói bàn giao được lưu thêm tại **/home/ubuntu/2026.1/HDH/BaoCao/Hoang_week01_2026-10-23/**. Các thay đổi chưa stage, chưa commit, chưa push; không sửa tài liệu yêu cầu gốc trong Doc. Review của Hiếu, tag GV, baseline máy Hoàng và triển khai async vẫn là việc chuyển tiếp.